NAZARBAYEV UNIVERSITY

Programming for Engineers

Week 8: Objects, Classes, Attributes, and Methods

Yanwei Wang
School of Engineering
Nazarbayev University

Fall 2026



Week 8 at a glance

Lecture 22

Objects, classes, and attributes

- Why OOP?
- Construct instances.
- Explain initialization and state.

Lecture 23

Methods and object behavior

- Explain `self`.
- Query and update state.
- Process lists of objects.

Lecture 24

Reliable engineering objects

- Validate proposed changes.
- Test state and behavior.
- Prepare for Assignment 4 and Lab 3.

Describe an entity → Give it state and behavior → Create instances → Test every change

Course scope: a first step into OOP

Official Session 8 content

- Advantages of object-oriented programming.
- Data attributes and methods.
- Instances of a class.
- Assignment 4 and Laboratory 3.

Kept for Session 9

- More on classes.
- Getters, setters, and information hiding.
- Class variables and inheritance.

How the examples are used

The sensor and material examples are teaching adaptations of the course book. Advanced class hierarchies, abstract classes, and NumPy are not needed for this week.

ENG 101 Course Specification, p. 4, Sessions 8–9; CLOs 1, 3, 6.

NAZARBAYEV UNIVERSITY

LECTURE 22

Objects, Classes, and Data Attributes

From a dictionary record to a named engineering type

NAZARBAYEV UNIVERSITY

Lecture 22 learning goals

1. Explain when grouping data and behavior is useful.
2. Distinguish a class, an instance, an attribute, and a method.
3. Define a small class and initialize it with `__init__`.
4. Explain `self` as the instance being used.
5. Create two instances and trace their separate data.
6. Read, modify, and test a simple object.

Central question

What information belongs to one particular engineering entity?

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Warm-up: a familiar sensor record

```
# A familiar Week 6 record can still be used with an
    ordinary function.
sensor = {"name": "Tank temperature", "unit": "deg C
    ", "value": 82.5}


def format_reading(record):
    # Data are supplied explicitly as the function
    argument.
    return f"{record['name']}: {record['value']:.1f}
     {record['unit']}"


print(format_reading(sensor))
```

Before running

1. Name the stored data.
2. Name the operation.
3. Predict the returned string.
4. Where is the sensor supplied?

Individual then pairs - 3 min

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Why introduce another way to organize code?

A laboratory has many entities

- Each sensor has a name, unit, and reading.
- Each has a rule for interpreting that reading.
- Each needs a readable report.

A useful class groups related ideas

- Data remain associated with one entity.
- Methods provide related operations.
- One definition can serve many instances.

Not a rejection of functions

Dictionaries plus well-designed functions are still an excellent choice for many small programs.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

You have already used objects and methods

```
# Strings and lists already provide methods
    associated with their types.
label = " tank "
readings = [82.5, 85.0]
print(label.strip())            # Return a new string
    .
readings.append(90.0)           # Change this list in
    place.
print(len(readings))            # A familiar function
    call.
print(readings)
```

Read the dot notation

- `label.strip()` asks a string for cleaned text.
- `readings.append(90.0)` changes a list.
- `len(readings)` is a function call.

Now create our own type

The next step is to give an engineering concept its own attributes and methods.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Four words that must stay distinct

Term	Meaning in this week	Example
Class	A definition of an object type and its operations	`Sensor`
Instance / object	One particular object created from that class	`temperature`
Data attribute	A named value associated with an instance	`temperature.value`
Method	An operation called using an instance	`temperature.report()`

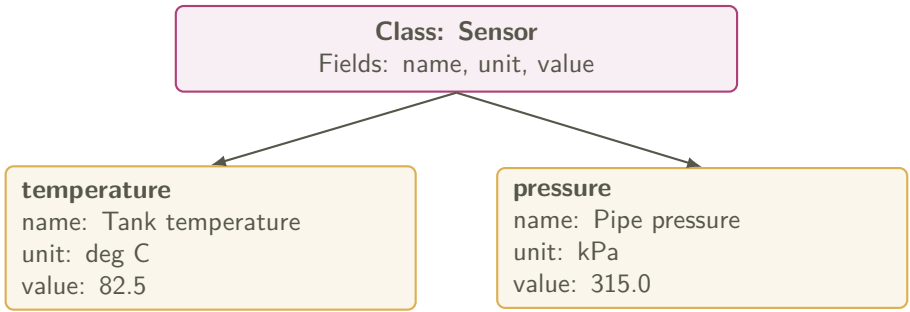
State and behavior

State is the current stored information. Behavior is what the object can calculate, answer, or change.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

One definition, several distinct instances



The engineering analogy

One design description; two specific devices with different current data.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

The smallest class: syntax before usefulness

```
# An empty class is a syntax demonstration, not our
    final data model.
class Sensor:
    pass                          # Placeholder: do
    nothing here.


s1 = Sensor()                     # Create one instance.
s1.name = "Tank temperature"      # Attach attributes
    manually.
s1.unit = "deg C"
s1.value = 82.5
```

Read each part

- `class Sensor:` introduces a class.
- `pass` is a placeholder statement.
- `Sensor()` creates an instance.
- `s1.value = 82.5` assigns an attribute.

Only a first step

Manually adding fields is fragile when every instance needs the same structure.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Manual fields can leave incomplete objects

A possible omission

Create two empty sensors, but forget to give the second one a `unit`. Trying to read that missing attribute raises `AttributeError`.

A repeatable initialization step

Require the expected information at object creation. Store the fields in one place, using the same names for every instance.

A design habit

Decide which attributes must exist before the object is used.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Initialize the expected fields in one place

```
class Sensor:
    """Store a sensor name, its unit, and its
     current value."""

    def __init__(self, name, unit, value):
        # Store each input on the instance being
     initialized.
        self.name = name
        self.unit = unit
        self.value = value
```

What the initializer receives

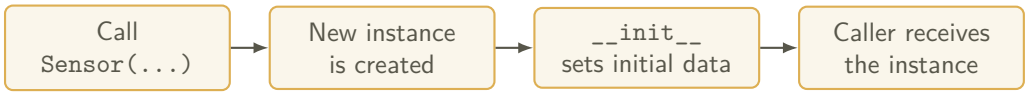
- `self`: the new instance.
- `name`, `unit`, `value`: caller arguments.

What it establishes

Every successfully initialized Sensor has a name, unit, and current value.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

Creating an instance is a sequence of events



An important distinction

`__init__` initializes the new object. It should not return the object or another non-`None` result.

Wang, Section 15.5; Python Tutorial, Section 9.3.2. Technical clarification of initialization.

NAZARBAYEV UNIVERSITY

The two sides of self.value = value

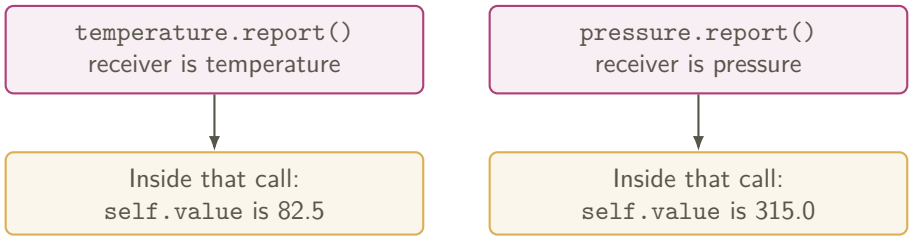
Expression	Meaning	Lifetime in this example
`value`	A local parameter name	Used during the call
`self.value`	An attribute on the receiver	Available through the object afterward
`self.value = value`	Store the supplied value on this instance	Makes its state explicit

The same spelling is not a requirement

`self.value = initial_value` is equally valid when the parameter is named `initial_value`.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

self means this particular instance



A naming convention

`self` is the conventional first parameter of an instance method, not a special Python keyword.

Wang, Section 15.6; Python Tutorial, Sections 9.3.4 and 9.4.

NAZARBAYEV UNIVERSITY

Create and inspect two sensors

```
temperature = Sensor("Tank temperature", "deg C",
    82.5)
pressure = Sensor("Pipe pressure", "kPa", 315.0)
print(temperature.name)
print(temperature.value, temperature.unit)
print(pressure.value, pressure.unit)
```

Expected output

```
Tank temperature
82.5 deg C
315.0 kPa
```

One type, different data

Both are Sensor instances, but each has its own name, unit, and value.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Attributes are accessed with a dot

Expression	Question or action
`temperature.name`	What label belongs to this sensor?
`temperature.unit`	Which unit is documented for the reading?
`temperature.value`	What is the stored value now?
`temperature.value = 85.0`	Replace this instance' s current reading.

Units still need engineering meaning

A text label does not automatically convert values or prevent mixing incompatible quantities.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

A local value is not a live link to the sensor

```
# Two calls create two separate instances.
a = Sensor("A", "deg C", 20.0)
b = Sensor("B", "deg C", 20.0)
a.value = 25.0                  # Change a, not b.
local_value = a.value           # Read the current
    numeric value.
local_value = 99.0              # Rebind a local name;
     a is unchanged.
print(a.value, b.value, local_value)
```

Predict three values

1. What is the final value in a?
2. What is the final value in b?
3. What is `local_value`?

Trace on paper - 4 min

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Trace the state after each assignment

Statement	a.value	b.value	local_value
Create a and b	20.0	20.0	not assigned
`a.value = 25.0`	25.0	20.0	not assigned
`local_value = a.value`	25.0	20.0	25.0
`local_value = 99.0`	25.0	20.0	99.0

Explain, do not merely remember

Identify which name or attribute is assigned on every line.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Inspect the type, not only the printed value

```
s1 = Sensor("Tank temperature", "deg C", 82.5)
print(type(s1).__name__)       # Name of the instance
    's type.
print(isinstance(s1, Sensor))  # Does this object
    have the expected type?
print(isinstance(s1.value, float))
```

Expected output

```
Sensor
True
True
```

Two levels of information

The sensor has type `Sensor`; its stored value can have type `float`.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

A dictionary and a class are both valid tools

Task	Dictionary record	Instance
Read the value	`record["value"]`	`sensor.value`
Change a value	`record["value"] = x`	`sensor.value = x`
Ask for a report	`report(record)`	`sensor.report()`
Best reason to choose	Simple named data with external functions	A meaningful entity with related operations

Choose by responsibility

Do not convert every dictionary into a class merely because classes are available.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Advantages come with responsibilities

Potential advantages

- Related state and operations stay together.
- A common interface can be reused.
- Each entity can be tested separately.
- The program can read like an engineering procedure.

Costs and risks

- More names and structure to learn.
- State changes can be less obvious.
- Aliases can cause unintended changes.
- A poorly chosen class adds complexity.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Live coding: a material data model

Design before typing

We need a material name and a constant density in kg/m^3. Define a class, initialize the attributes, and create two materials.

```
class Material:
    """Store a name and an illustrative constant
     density in kg/m^3."""

    def __init__(self, name, density_kg_m3):
        # This introductory model assumes positive
     finite density.
        self.name = name
        self.density_kg_m3 = density_kg_m3
```

Student task

1. Create Aluminum with density 2700.
2. Create Steel with density 7850.
3. Print each name and density.

Build and inspect - 4 min

The densities are illustrative values used for programming practice, not design specifications.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Test an instance immediately after creation

```
# These tests inspect the state after initialization.
s = Sensor("Test", "deg C", 20.0)
assert s.name == "Test"
assert s.unit == "deg C"
assert s.value == 20.0
print("Initialization checks passed")
```

Week 7 connection

Tests should confirm that the initialized state matches the specification.

What is not proved?

These checks do not validate a real device, a sensor calibration, or every possible input.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Debugging clinic: local name or attribute?

```
# Intentional error: value is local, not an instance
    attribute.
class Sensor:
    def __init__(self, initial_value):
        value = initial_value

s = Sensor(82.5)
print(s.value)                  # No assignment to self
    .value took place.
```

Diagnose first

1. Does the initializer run?
2. Which name receives the input?
3. Does s.value exist?
4. What is the smallest repair?

Pairs - 3 min

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

A first-class design checklist

1. Give the class one clear engineering responsibility.
2. Choose a meaningful class name and instance names.
3. Document each attribute, its meaning, and its unit.
4. Use `__init__` to establish the expected initial state.
5. Create two instances to test that their data are distinct.
6. Test what the caller should observe, not just that code runs.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

Lecture 22 exit ticket

A resistor data model

1. Write a class name for one ideal resistor.
2. Choose an attribute storing resistance in ohms.
3. Write its initializer.
4. Create 4-ohm and 8-ohm instances.

Explain aloud

What does `self` refer to? Why are the two instances not the same object?

Individual - 3 min

Next lecture

Give an instance useful behavior: calculations, queries, updates, and reports.

Wang, Ch. 15, Sections 15.1–15.7; Guttag, Section 8.1. Teaching adaptations.

NAZARBAYEV UNIVERSITY

LECTURE 23

Methods and Object Behavior

Query state, update deliberately, and combine familiar tools

NAZARBAYEV UNIVERSITY

Lecture 23 learning goals

1. Define and call an ordinary instance method.
2. Explain why `self` is supplied automatically at the call site.
3. Distinguish calculation, query, update, and report methods.
4. Trace returned values and changes to stored state.
5. Process and filter a list of sensor instances.
6. Recognize missing calls, missing receivers, and aliasing.

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Warm-up: identify the receiver

Expression	Receiver	What kind of result?
`label.strip()`	`label`	A cleaned string
`readings.append(90.0)`	`readings`	A changed list; the return is None
`sensor.report()`	`sensor`	A report string in our next class

Discuss

Which part before the dot selects the object? Why is the returned result a separate question?

Pairs - 3 min

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

A method uses its receiver's data

```
class Sensor:
    def __init__(self, name, unit, value):
        self.name = name
        self.unit = unit
        self.value = value

    def report(self):
        # Return text so the caller decides where to
    display it.
        return f"{self.name}: {self.value:.1f} {self
    .unit}"
```

The method body

- `self.name` selects this instance's label.
- `self.value` selects its current reading.
- `return` sends text to the caller.

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

Why does the call omit self?

```
s = Sensor("Tank temperature", "deg C", 82.5)
print(s.report())
print(Sensor.report(s))        # Equivalent call for
    this ordinary method.
```

For this ordinary instance method

`s.report()` supplies `s` as the first argument. It has the same effect here as `Sensor.report(s)`.

Preferred everyday syntax

Use `s.report()`. The explicit class call is shown only to explain binding.

Wang, Section 15.6; Python Tutorial, Section 9.3.4. Receiver-binding clarification.

NAZARBAYEV UNIVERSITY

Read data; call behavior

Expression	Meaning
`s.value`	Read a stored number
`s.report`	Obtain the method object without running it
`s.report()`	Run the method and obtain its returned text
`s.value()`	Usually an error here: a numeric value is not callable

A familiar Week 3 mistake

Missing parentheses can leave you holding a method rather than the result you intended to calculate.

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Give the sensor a clear interval contract

Attribute	Meaning
`name`, `unit`	Human-readable label and measurement unit
`value`	Current finite numeric reading
`lower_limit`	Lower acceptable boundary, included
`upper_limit`	Upper acceptable boundary, included

Assumptions for this first model

The caller supplies ordered limits and compatible numeric values in the same unit. Lecture 24 adds explicit validation.

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Initialize an instance with its own limits

```
class Sensor:
    """Bundle a current reading, units, limits, and
    related operations."""

    def __init__(self, name, unit, value,
    lower_limit, upper_limit):
        # Each call initializes one instance, not a
    shared reading.
        self.name = name
        self.unit = unit
        self.value = value
        self.lower_limit = lower_limit
        self.upper_limit = upper_limit
```

A complete call

```
Sensor("Tank", "deg C", 82.5,
0.0, 90.0)
```

Do not confuse data and rules

The reading can change while the operating limits remain the same.

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

A query method answers a Boolean question

```
def is_safe(self):
    """Return whether the reading is within the
 stated inclusive limits."""
    return self.lower_limit <= self.value <=
 self.upper_limit
```

The rule

$$\text{lower limit} \leq \text{value} \leq \text{upper limit}.$$

A boundary value is included.

Name versus guarantee

Here `is_safe()` means only "within the stated limits. It does not certify a physical system.

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

An update method changes state deliberately

```
    def update(self, new_value):
        """Replace the reading; this first model
     assumes valid numeric input."""
        self.value = new_value
        # No explicit value is returned: the call
     returns None.
```

Calling the update

`s.update(95.0)` stores a new current reading on `s`. The old value is replaced.

No value returned

This update method returns `None`. Do not write `s = s.update(95.0)`.

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

A report can call a query method

```
def report(self):
    """Return a labeled result without changing
 the stored reading."""
    if self.is_safe():
        status = "WITHIN LIMITS"
    else:
        status = "OUTSIDE LIMITS"
    return (f"{self.name}: {self.value:.1f} {
 self.unit} | "
            f"{status}")
```

One responsibility per operation

- `is_safe()` evaluates the rule.
- `report()` constructs a message.
- The caller uses `print()` to display it.

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

Return and print are still different

```
s = Sensor("Tank temperature", "deg C", 82.5, 0.0,
    90.0)
text = s.report()               # A string is returned
    to the caller.
print(text)                     # Display it once.
result = s.update(95.0)         # Mutate the reading;
    return None.
print(result)
print(s.value)
```

Output

```
Tank temperature: 82.5 deg C |
    WITHIN LIMITS
None
95.0
```

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Test both boundaries, not one convenient value

```
s = Sensor("Temperature", "deg C", 50.0, 0.0, 90.0)
# Both interval endpoints are included in the
    teaching rule.
for value in [-0.1, 0.0, 50.0, 90.0, 90.1]:
    s.update(value)
    print(f"{s.value:5.1f}: {s.is_safe()}")
```

Observed results

```
-0.1: False
 0.0: True
50.0: True
90.0: True
90.1: False
```

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Trace a reading before and after an update

```
s = Sensor("Tank temperature", "deg C", 82.5, 0.0, 90.0)
print(s.report())
s.update(95.0)                # Only this instance's current value changes.
print(s.report())
```

Output

```
Tank temperature: 82.5 deg C | WITHIN LIMITS
Tank temperature: 95.0 deg C | OUTSIDE LIMITS
```

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Four kinds of method, four different contracts

Kind	Example	Changes state?	Returns
Calculation	`material.mass_from_volume(v)`	No	A number
Query	`sensor.is_safe()`	No	A Boolean
Update	`sensor.update(x)`	Yes	None here
Report	`sensor.report()`	No	A string here

Before using any method

Ask what it reads, what it changes, and what its call returns.

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Prediction: old result or current state?

```
s = Sensor("Test", "deg C", 82.5, 0.0, 90.0)
answer = s.is_safe()          # Compute the result
    now.
s.update(95.0)                # Change the object
    later.
print(answer)                 # Stored bool does not
    update itself.
print(s.is_safe())            # A new call uses the
    current state.
print(s.value)
```

Predict every line

1. When is answer calculated?
2. Does it change after update?
3. What does the new query return?

Individual then pairs - 4 min

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

A method reference is not a decision result

```
s = Sensor("Tank temperature", "deg C", 95.0, 0.0,
    90.0)
check = s.is_safe              # BUG: store the method
    , not its result.
if check:                     # A bound method is
    truthy.
    print("WRONG branch: method was not called")
print(s.is_safe())            # Correct evaluation
    returns False.
```

Find the defect

`check = s.is_safe` does not call the method. A method object is truthy even when calling it would return False.

Repair

Use `check = s.is_safe()` or directly `if s.is_safe():`.

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

A material can calculate with its stored density

```
class Material:
    """Store a name and an illustrative constant
    density in kg/m^3."""

    def __init__(self, name, density_kg_m3):
        # This introductory model assumes positive
    finite density.
        self.name = name
        self.density_kg_m3 = density_kg_m3

    def mass_from_volume(self, volume_m3):
        """Return mass in kg; assume a finite
    nonnegative volume in m^3."""
        return self.density_kg_m3 * volume_m3
```

Engineering model

$$m = \rho V.$$

Density belongs to the material; the part volume is supplied for each call.

Scope

Constant, uniform density and consistent units are assumed.

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Check the calculation independently

For the illustrative density $\rho = 2700$ kg/m^3 and volume $V = 0.002$ m^3:

$$m = 2700 \times 0.002 = 5.4 \text{ kg}.$$

```
# Illustrative constant densities, not certified design-property data.
aluminum = Material("Aluminum", 2700.0)
steel = Material("Steel", 7850.0)
volume_m3 = 0.002
print(f"Aluminum mass = {aluminum.mass_from_volume(volume_m3):.2f} kg")
print(f"Steel mass = {steel.mass_from_volume(volume_m3):.2f} kg")
assert abs(aluminum.mass_from_volume(volume_m3) - 5.4) < 1e-12
```

Output

```
Aluminum mass = 5.40 kg
Steel mass = 15.70 kg
```

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

Objects fit inside familiar lists

```
# Each list element refers to a different Sensor
    instance.
sensors = [
    Sensor("Tank temperature", "deg C", 82.5, 0.0,
     90.0),
    Sensor("Pipe pressure", "kPa", 315.0, 100.0,
     400.0),
    Sensor("Battery voltage", "V", 10.8, 11.0, 12.8)
     ,
]
for sensor in sensors:
    print(sensor.report())
```

Same loop, richer elements

- Each element refers to one Sensor.
- The loop variable selects the current object.
- Its methods use that object' s own state.

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Filter objects without losing their metadata

```
# Filter references to objects, not only their
    numeric values.
flagged = []
for sensor in sensors:
    if not sensor.is_safe():
        flagged.append(sensor)
for sensor in flagged:
    print(sensor.name)
```

Retain the whole entity

The flagged list keeps references to Sensor objects, so labels, units, limits, and methods remain available.

Result

Battery voltage is the only flagged sensor in the supplied three-object example.

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Aliasing did not disappear when we made a class

```
original = Sensor("Tank", "deg C", 82.5, 0.0, 90.0)
backup = original              # Another name, not
    another Sensor.
backup.update(95.0)
print(original.value, backup.value)
print(original is backup)     # Test identity: same
    object?
```

Predict the output

```
95.0 95.0
True
```

Why?

`backup = original` creates another name for the same object, not another sensor.

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

A new instance can preserve a snapshot

```
original = Sensor("Tank", "deg C", 82.5, 0.0, 90.0)
# This explicit new instance is sufficient for these scalar/
    string fields.
snapshot = Sensor(original.name, original.unit, original.
    value,
                  original.lower_limit, original.
    upper_limit)
original.update(95.0)
print(original.value, snapshot.value)
print(original is snapshot)
```

Output

```
95.0 82.5
False
```

This explicit snapshot is sufficient for these scalar and string fields; nested mutable data require a deliberate copying policy.

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Debugging clinic: find the receiver mistakes

Missing receiver parameter

```
class Sensor:
    def report():
        return "Reading"

s = Sensor()
s.report()                    # Python supplies an
    instance argument.
```

Class called without an instance

```
class Sensor:
    def report(self):
        return "Reading"

Sensor.report()               # No
    instance is supplied as self.
```

Repair before running

Supply self in the method definition; call the method through a specific instance.

Pairs - 3 min

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Lecture 23 exit ticket

1. Explain `sensor.value`, `sensor.report`, and `sensor.report()`.
2. What should `sensor.update(95.0)` change and return in our class?
3. Why can `backup = sensor` change the original through another name?
4. Propose one boundary test and one independent-instance test.

Individual - 3 min

Today in one sentence

A method acts on a selected instance; its contract must make state changes and returned results clear.

Wang, Ch. 15, Sections 15.6–15.9, 15.11, 15.15–15.16. Teaching adaptations.

NAZARBAYEV UNIVERSITY

LECTURE 24

Reliable Engineering Objects

Validation, tests, and a small object-based monitoring workflow

NAZARBAYEV UNIVERSITY

Lecture 24 learning goals

1. Write a simple object-state contract before adding checks.
2. Separate invalid data from valid readings outside a limit.
3. Validate a proposed update before changing stored state.
4. Test initialization, boundaries, failures, and independence.
5. Combine ordinary functions, objects, and loops in one workflow.
6. Plan a small class-based task collaboratively.

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

Warm-up: invalid or outside the operating limit?

Absolute pressure	Limit	Expected treatment
315 kPa	400 kPa	Store it; within limit
400 kPa	400 kPa	Store it; boundary included
900 kPa	400 kPa	Store it; report limit exceeded
-5 kPa	400 kPa	Reject under this absolute-pressure contract

Discuss

Why would silently discarding a high but valid reading be misleading?

Pairs - 3 min

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

Decide the responsibility before the methods

One PressureSensor instance

- Stores a label and a current absolute pressure.
- Stores an inclusive warning threshold.
- Accepts or rejects proposed readings.
- Returns a report using the stored state.

Not part of this class

- Reading from physical hardware.
- Controlling a valve or stopping a process.
- Certifying that a system is safe.
- Implementing every type of sensor.

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

Write the input and state contract

Item	Teaching contract
Quantity and unit	Absolute pressure in kPa; one fixed unit convention
Numeric inputs	Built-in int or float; convertible to finite float; not bool
Physical domain	Pressure and the chosen threshold are nonnegative
Operating rule	`value_kpa <= limit_kpa`
Failed update	Raise an appropriate exception; preserve the old reading

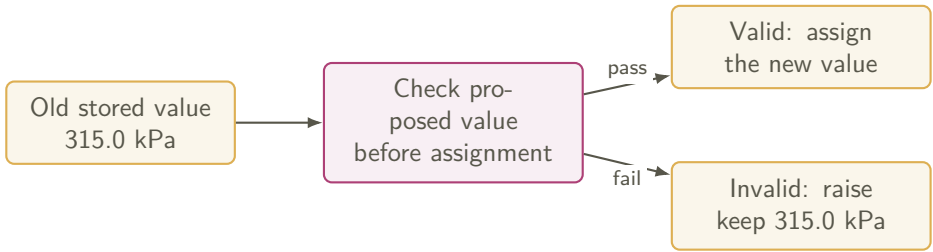
Adaptation from the reading

The explicit finite-number and type checks extend the course-book pressure example using Week 7 tools.

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

Validate before committing a state change



A useful failure guarantee

A failed update should not leave this object with a partially changed or invalid reading.

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

Reuse a checked-number function

Helper interface

`checked_nonnegative(value, label)`

- Returns a validated float.
- Rejects wrong types with TypeError.
- Rejects nonfinite or negative values with ValueError.
- Uses label to make errors understandable.

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

Why a function outside the class?

The same check is useful for the initial value, threshold, and later updates. Functions and classes work together.

Source available

The complete helper is included in the appendix and the Python package.

Initialize a checked pressure object

```
class PressureSensor:
```

```
    def __init__(self, name, value_kpa, limit_kpa):
        # Check the label and both numeric inputs at initialization.
        if not isinstance(name, str) or not name.strip():
            raise ValueError("name must be nonempty text")
        self.name = name.strip()
        self.unit = "kPa (absolute)"
        self.limit_kpa = checked_nonnegative(limit_kpa, "limit_kpa")
        self.value_kpa = checked_nonnegative(value_kpa, "value_kpa")
```

Inside class PressureSensor

The helper is called for both numeric fields. An invalid initial reading prevents successful initialization.

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

The checked update is still a small method

```
    def update(self, new_value_kpa):
        """Store a valid reading; on failure, leave
     the old state unchanged."""
        # Validation finishes before the assignment
     can change the state.
        checked = checked_nonnegative(new_value_kpa,
      "new_value_kpa")
        self.value_kpa = checked
```

Execution order

1. Check the proposed value.
2. Receive the validated number.
3. Assign it to the instance.

Failure path

If validation raises an exception, the assignment is not reached.

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

A query should report the current state

```
    def is_safe(self):
        """Return True only when absolute pressure is no greater than the limit."""
        return self.value_kpa <= self.limit_kpa

    def report(self):
        """Return the reading, criterion, and result; do not operate hardware."""
        if self.is_safe():
            status = "WITHIN LIMIT"
        else:
            status = "LIMIT EXCEEDED"
        return (f"{self.name}: {self.value_kpa:.1f} {self.unit} | "
                f"limit={self.limit_kpa:.1f} kPa | {status}")
```

Keep measurements visible

A high but valid pressure is stored and then reported as exceeding the selected limit.

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

A high reading is not a parsing error

```
# This class models absolute pressure; do not apply it to gauge readings.
p = PressureSensor("Pipe", 315.0, 400.0)
print(p.report())
p.update(900.0)                # A valid reading above the chosen limit.
print(p.report())
```

Output

```
Pipe: 315.0 kPa (absolute) | limit=400.0 kPa | WITHIN LIMIT
Pipe: 900.0 kPa (absolute) | limit=400.0 kPa | LIMIT EXCEEDED
```

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

Test a normal value and an exact threshold

```
p = PressureSensor("Test", 0.0, 400.0)
assert p.is_safe()            # Zero is valid for
    this mathematical model.
p.update(400.0)
assert p.is_safe()            # Inclusive upper
    boundary.
p.update(400.1)
assert not p.is_safe()
assert p.value_kpa == 400.1   # Do not hide an
    exceeded limit.
print("Pressure value and boundary checks passed")
```

Expected outcome

```
Pressure value and boundary
    checks passed
```

More than testing a return

The last assertion verifies that the exceeded-limit reading was actually retained.

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

Test the state after a failed update

```
p = PressureSensor("Pipe", 315.0, 400.0)
before = p.value_kpa
try:
    p.update(-5.0)             # Rejected by the
     absolute-pressure contract.
except ValueError:
    print("Rejected invalid absolute pressure")
assert p.value_kpa == before  # Failure must not
     corrupt the stored state.
print(f"Stored reading = {p.value_kpa:.1f} kPa")
```

Expected output

```
Rejected invalid absolute
    pressure
Stored reading = 315.0 kPa
```

Check both effects

The invalid input is rejected, and the original value remains stored.

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

Use an expected-failure test, not silence

A complete failure test must establish two facts

1. The operation raised the documented exception.
2. The stored state still equals its previous value.

A common testing gap

Checking the old state alone would also pass if a broken update silently did nothing. Both the failure and the preserved state matter.

Explain the missing test - 2 min

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

Test that separate instances stay separate

```
first = PressureSensor("A", 100.0, 400.0)
second = PressureSensor("B", 100.0, 400.0)
first.update(450.0)
assert first.value_kpa == 450.0
assert second.value_kpa == 100.0
assert first is not second
print("Independent-instance checks passed")
```

What the test demonstrates

- Two construction calls create two objects.
- Updating first does not update second.
- `is not` confirms different identities.

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

Combine a method with familiar exception handling

```
p = PressureSensor("Pipe", 315.0, 400.0)
# Fixed teaching inputs make this demonstration reproducible.
for proposed_kpa in [315.0, 400.0, 900.0, -5.0, 380.0]:
    try:
        p.update(proposed_kpa)
    except (TypeError, ValueError) as error:
        print(f"REJECTED {proposed_kpa}: {error}")
    else:
        print(p.report())      # Report only a successfully accepted reading.
```

One reading per loop pass

The caller tries an update, reports an accepted reading, and identifies a rejected input explicitly.

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

Trace the complete monitoring sequence

Proposed kPa	Accepted?	Stored afterward	Operating result
315.0	Yes	315.0	Within limit
400.0	Yes	400.0	Within limit
900.0	Yes	900.0	Limit exceeded
-5.0	No	900.0	Rejected; old value retained
380.0	Yes	380.0	Within limit

Distinguish the events

A rejected proposal is not a new measurement stored in the object. Preserve its rejection message separately.

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

Objects still need a test matrix

Case	Expected result	Evidence to retain
Initial valid reading	Object has expected fields	Inspect name, unit, value, limit
Exact operating limit	Query returns True	Boundary check
Above operating limit	Stored; query returns False	Reading plus classification
Invalid proposal	Exception; old state unchanged	Exception type and before/after values
Two objects	Only the selected object changes	Independent-instance check

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

Public attributes are not automatically protected

Documented route

`p.update(-5.0)` is checked and rejected before storing the new value.

Direct assignment

`p.value_kpa = -5.0` bypasses that method in this plain Python class.

A boundary of this week' s design

Use the documented interface. Week 9 develops information hiding, getters/setters, and further class design.

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

When is a class worth using?

A class can help

- One recognizable entity owns persistent state.
- Several operations depend on that state.
- Many instances share the same interface.
- The design becomes easier to explain and test.

A function may be enough

- One formula with explicit inputs.
- A short one-off conversion.
- No persistent state or entity is needed.
- A class would mostly wrap a single line.

Engineering rule of thumb

Choose the simplest organization that keeps meaning, assumptions, and evidence visible.

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

Pair design activity: one entity, clear contracts

Choose a small engineering entity

An ideal resistor, a material record, or a temperature sensor.

1. List its data attributes with units.
2. Propose one calculation or query method.
3. Propose one update only if it is genuinely needed.
4. Specify one known-answer test and one failure case.
5. Exchange the interface with another pair and review it.

Design then peer review - 5 min

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

Assignment 4: prepare to work with objects

Preparation questions

- Name the entity and its state.
- Specify returned values and changes.
- Prepare known-answer and boundary tests.

Before submitting assessed work

- Read the final task and marking criteria.
- Check permitted tools and collaboration.
- Explain your code and document assistance as required.

Authoritative brief

Assignment 4 is listed in Session 8. Follow Moodle for the final task, deadline, grading, and submission rules.

ENG 101 Course Specification, Session 8 and E-Learning section; OOP preparation guidance.

NAZARBAYEV UNIVERSITY

Laboratory 3: a small practice workflow

Illustrative practice data

Inclusive range: 20–$30\,^{\circ}\mathrm{C}$. Readings: 19.5, 20.0, 25.0, 30.0, 30.5.

Workflow

1. Create one Sensor.
2. Update and report each reading.
3. Count readings outside the range.

Checkable results

- Two warnings.
- Both boundaries included.
- Final stored value: 30.5.

Moodle is authoritative

This is practice, not the final Laboratory 3 assessment brief.

Course Specification, Session 8; practice adaptation of Wang, Sections 15.9 and 15.16.

NAZARBAYEV UNIVERSITY

Review the design, not only the output

1. Does the class have one clear purpose?
2. Are attribute names, units, and method responsibilities explicit?
3. Does each query use the current instance rather than hidden global state?
4. Do checked updates preserve the old state when they reject input?
5. Do tests include boundaries, wrong inputs, and a second instance?
6. Does the report show both the reading and the decision criterion?

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

Lecture 24 exit ticket

A proposed update fails

A PressureSensor stores 315.0 kPa. The call `update(-5.0)` is made. Explain:

1. Which input contract is violated?
2. What should happen to the stored value?
3. Which exception is expected for this value?
4. How would you test all three claims?

Individual - 3 min

Forward connection

Next week: more on classes, controlled attribute access, class variables, and inheritance.

Wang, Ch. 15, Sections 15.10, 15.16–15.17, 15.26–15.27. Week 7 testing connection.

NAZARBAYEV UNIVERSITY

Reference: the complete numeric helper

```
def checked_nonnegative(value, label):
    if isinstance(value, bool) or not isinstance(value, (int, float)):
        raise TypeError(f"{label} must be an int or float, not bool")
    try:
        number = float(value)
    except OverflowError as error:
        raise ValueError(f"{label} is too large for this model") from error
    if not math.isfinite(number) or number < 0:
        raise ValueError(f"{label} must be finite and nonnegative")
    return number
```

Library support

`math.isfinite(number)` rejects NaN and infinities. This helper intentionally accepts only built-in int/float inputs, not bool or numeric strings.

Week 7 validation pattern; Python math.isfinite documentation. Explicit robustness extension.

Reference: each instance can own a list

```
class ReadingLog:
    def __init__(self, name):
        self.name = name
        self.readings = []

    def add(self, value):
        # Teaching version assumes a valid numeric
     reading.
        self.readings.append(value)
```

Demonstration output

```
[82.5]
[]
```

Why it works

`self.readings = []` executes for each instance. Each receives a fresh list.

Wang, Appendix H, Sections H.8 and H.10; transfer from Week 5 mutability.

Reference: receiving a list is not copying it

Alias the supplied list

```
class ReadingLog:
    def __init__(self, readings):
        self.readings = readings
```

The caller and the attribute refer to the same list.

Copy the supplied outer list

```
class ReadingLog:
    def __init__(self, readings):
        self.readings = readings.copy
    ()
```

For numeric readings, this preserves the caller' s outer list.

Copying policy

A shallow copy does not recursively copy nested mutable objects. Choose and document the intended relationship.

Wang, Section 5.10 and Appendix H; explicit application of aliasing to attributes.

Worked answer: an ideal resistor class

```
class Resistor:
    def __init__(self, resistance_ohm):
        if resistance_ohm <= 0:
            raise ValueError("Resistance must be
     positive")
        self.resistance_ohm = resistance_ohm

    def current_a(self, voltage_v):
        # Ohm's law for an ideal linear resistor: I =
      V/R.
        return voltage_v / self.resistance_ohm
```

Two known-answer tests

`Resistor(4.0).current_a(12.0)` returns 3.0 A.

`Resistor(4.0).current_a(0.0)` returns 0.0 A.

Limitations

Assume finite numeric inputs and an ideal linear resistor. This short solution is not a full type-validation library.

Wang, Appendix H, instance-method examples; worked teaching exercise using Ohm' s law.

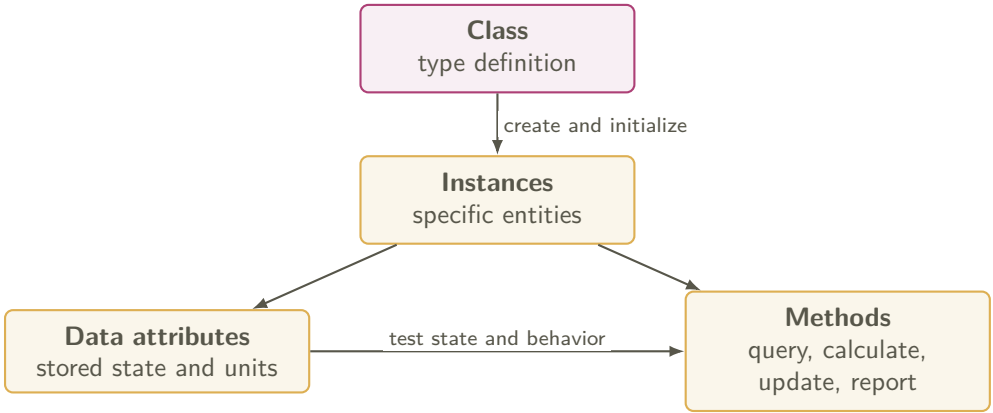
NAZARBAYEV UNIVERSITY

Reference: common class mistakes

Symptom	Likely cause	Check
Missing expected attribute	Local `value`, not `self.value`	Initialization assignment
Wrong argument-count error	Missing `self` or missing receiver	Definition and call
Wrong branch with no error	Method referenced but not called	Missing parentheses
Two names change together	Both names refer to one instance	Aliasing
Initialization rejects a return	Non-None result returned by `__init__`	Remove value-return statement

Wang, Section 15.27; Python Tutorial, Section 9.3; executable debugging exercises.

Week 8 concept map



Synthesis of Course Specification Session 8 and Wang, Chapter 15.

NAZARBAYEV UNIVERSITY

Suggested reading and source basis

- **Course scope:** Nazarbayev University, *ENG 101 Course Specification Report*, Session 8; Session 9 for the next scope boundary.
- **Engineering examples:** Y. Wang, *Programming for Engineers with Python*, supplied working manuscript, Sections 15.1–15.11, 15.15–15.17, 15.26–15.27.
- **Data abstraction:** J. V. Guttag, *Introduction to Computation and Programming Using Python*, 2nd ed., Section 8.1 (inheritance deferred).
- **Python semantics:** Python Tutorial, Sections 9.3–9.4; official classes documentation.

How to study

Predict a call, draw the instance state, run the small example, and explain what changed.

Primary course materials supplied by the instructor; Python documentation used for technical checks.

NAZARBAYEV UNIVERSITY

Source adaptations and scope notes

- `Sensor.report()` returns text here rather than printing it, preserving Week 4 separation of calculation and display.
- The source name `is_safe()` is retained, but reports say "within limits rather than imply certified safety.
- The checked pressure example adds finite-number checks and a failed-update test using Week 7 ideas.
- Illustrative data and practice tasks are not official Assignment 4 or Laboratory 3 requirements.
- Inheritance, class variables, and detailed information hiding remain for Week 9.

Adaptation record for the Week 8 teaching package.

NAZARBAYEV UNIVERSITY

Objects organize engineering meaning.

Keep the data meaningful.
Make the behavior explicit.
Test every important change.

ENG 101 – Programming for Engineers